

TypeScript – En komplett referensguide

Från grundläggande typer till typsäkra API-anrop och DOM-hantering

TypeScript (TS) är ett programmeringsspråk som bygger ovanpå JavaScript. Man brukar kalla det för en *superset* (överbild) till JavaScript. Det betyder i praktiken att all vanlig JavaScript-kod också är giltig TypeScript-kod, men att TypeScript lägger till ett kraftfullt extra lager av säkerhet: **statisk typning**.

Webbläsare förstår inte TypeScript direkt. Därför körs koden genom en kompilator eller ett modernt byggverktyg (som `Vite`) som rensar bort alla typ-definitioner och översätter koden till vanlig JavaScript.

1. De Primitiva Typerna

Detta är de mest grundläggande datatyperna som vi använder för att definiera enskilda variabler:

```
// Explicit typning (vi talar om typen direkt)
let brand: string = "Volvo";
let year: number = 1983;
let isRetro: boolean = true;
let carId: undefined = undefined;

// Implicit typning (TypeScript gissar typen själv utifrån värdet - helt okej!)
let model = "240 Turbo"; // TS förstår automatiskt att detta är en string
```

2. Arrayer (Samlingar)

När du vill lagra och hantera listor med data av en och samma specifika typ:

```
// En array som bara får innehålla strängar
let colors: string[] = ["Röd", "Blå", "Silver"];

// En array som bara får innehålla tal
let years: number[] = [1983, 1999, 2026];

// ✗ Detta ger felmeddelande i VS Code direkt:
colors.push(123); // Argument of type 'number' is not assignable to parameter of type 'string'.
```

3. Interface (Datamodeller)

Ett `interface` definierar ett "kontrakt" eller en exakt ritning för hur ett objekt måste se ut. Det är det perfekta verktyget för att spegla dina C#-modeller (DTO:er).

```
interface ICar {
    id?: number; // ? betyder valfri (optional). Perfekt om ID sätts först i databasen.
    brand: string;
    model: string;
    year: number;
    color: string;
}

// ● Godkänt objekt: Följer interfacet till punkt och pricka
const retroCar: ICar = {
    brand: "Saab",
    model: "900",
    year: 1988,
    color: "Blå"
};

// ✗ Felaktigt objekt: Saknar 'color' och har felstavat 'year'
const brokenCar: ICar = {
    brand: "Ford",
    model: "Sierra",
    yar: 1991 // Fel! Property 'yar' does not exist on type 'ICar'.
};
```

4. Union Types (Flera tillåtna typer)

Ibland måste en variabel tillåtas acceptera mer än en typ av data. Då använder vi vertikalstreck (|).

```
// Ett ID från en databas kan vara ett heltal eller en unik GUID-sträng
let dbId: number | string;

dbId = 42; // ● OK!
dbId = "abc-12345"; // ● OK!
dbId = true; // ✗ Fel! Boolean är inte tillåten.
```

5. Funktioner (Parametrar & Returtyper)

I TypeScript deklarerar vi alltid vad en funktion förväntar sig att ta emot och vad den ska leverera tillbaka.

```
// En funktion som tar emot en bil och returnerar en formaterad sträng
function getCarDescription(car: ICar): string {
    return `${car.brand} ${car.model} från ${car.year}`;
}

// En funktion som inte returnerar något alls använder 'void'
function logMessage(message: string): void {
    console.log(`[LOGG]: ${message}`);
    // Ingen return-sats här
}
```

6. DOM-element & Type Casting (as)

När du hämtar element från HTML vet inte TypeScript vilket specifikt element det rör sig om. Vi använder `as` för att berätta för kompilatorn exakt vad det är.

```
// 1. Hämta en knapp
const submitBtn = document.querySelector('#submit-btn') as HTMLButtonElement;

// 2. Hämta ett inmatningsfält (så vi kan läsa av .value utan felmeddelanden)
const brandInput = document.querySelector('#brand-input') as HTMLInputElement;
const userText = brandInput.value; // ● Nu vet TS att .value existerar!

// 3. Hämta en container (div)
const carContainer = document.querySelector('#car-list') as HTMLDivElement;
```

7. Hantera fetch med Typsäkerhet

När vi hämtar data från ett API kan vi styra upp datan så att den matchar våra modeller direkt vid uppackningen.

```
const API_URL = "https://localhost:7187/api/cars";

const fetchCars = async (): Promise<void> => {
  try {
    const response = await fetch(API_URL);

    if (!response.ok) {
      throw new Error(`Nätverksfel: ${response.status}`);
    }

    // Vi talar om för TS att det vi packar upp (json) är en array av bilar: ICar[]
    const cars: ICar[] = await response.json();

    // Nu vet VS Code exakt vilka egenskaper som finns i "cars" när vi loopar!
    cars.forEach(car => {
      console.log(car.brand); // Fullt stöd för autoslutförande (IntelliSense)
    });

  } catch (error) {
    console.error("Kunde inte hämta bilar:", error);
  }
};
```

💡 Tre viktiga tips vid utveckling

- **Undvik `any`** : Att sätta typen till `any` stänger av alla TypeScript's kontroller. Använd det bara som absolut sista nödutgång om du har fastnat helt.
- **Läs felmeddelandena aktivt**: TypeScript är inte din fiende, det är din kompis. Om en röd linje dyker upp, hovra över den med muspekaren – felmeddelandet beskriver nästan alltid exakt vad som saknas eller blivit fel.
- **Kolla Network-fliken i utvecklarverktygen**: Om din kod kraschar på `JSON.parse` beror det ofta på att ditt WebAPI svarade med ren text (eller HTML-kod vid krasch) eller en tom body (som vid statuskod `204 No Content`), vilket inte kan omvandlas till ett JSON-objekt.